

종합과제

TDD TV Channel Controller

VSCode Java & C++ 개발자 과정 — Test Double(Fake/Mock) 활용 TDD 구현

★ C++ (Google Test / GMock) 위주 · Java (JUnit 5 / Mockito) 참고 대응

Java 17

C++17

VSCode

JUnit 5

Google Test

Mockito

GMock

Fake Object

TDD

과제 개요

프로젝트명	TDD_TV_Init
Repository	TDD_TV_XX (개인번호)
브랜치 구조	main → dev → refactoring
Reviewer	미작성 (개인 평가)
Merge	금지 - PR 만 생성
언어	Java 17 / C++17 선택 또는 병행
테스트 FW	JUnit 5 / Google Test
Test Double	Fake 또는 Mockito / GMock

평가 목표

✓ Coverage 높은 테스트 작성

채널 변경·선호채널·다음선호채널의 경계값·예외 케이스를 빠짐없이 커버합니다.

✓ 효과적인 Test Case 설계

Given-When-Then 구조로 읽기 쉬운 테스트를 작성하고 커밋 히스토리로 TDD 과정을 보여줍니다.

✓ Test Double 활용

실제 Tuner 대신 Fake/Mock을 사용해 빠르고 독립적인 단위 테스트를 만듭니다.

✓ TDD 중간 과정 커밋

Red → Green → Refactor 사이클이 커밋에 드러나도록 자주 push 합니다.

main

초기 소스 (수정 금지)

dev

개발 통합 브랜치

refactoring

리팩토링 작업 브랜치

❏ C++ - Git / CMakeLists.txt

```
# — 공통 Git 설정 (C++ / Java 동일) —————
git clone <TDD_TV_Init URL>
cd TDD_TV_Init
git checkout -b dev          # dev 브랜치 생성
git checkout -b refactoring  # refactoring 브랜치

# TDD 사이클마다 커밋 (커밋이 TDD 증거!)
git add . && git commit -m "test: 숫자버튼_채널변경_Red"
git push origin refactoring

# — C++ CMake 빌드 & 테스트 —————
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j4
cd build && ctest --output-on-failure

# — CMakeLists.txt (핵심 설정) —————
cmake_minimum_required(VERSION 3.14)
project(TDD_TV_CXX)
set(CMAKE_CXX_STANDARD 17)

include(FetchContent)
FetchContent_Declare(googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
  GIT_TAG        v1.14.0)
FetchContent_MakeAvailable(googletest)

add_executable(tv_test
  test/TVChannelControllerTest.cpp
  src/TVChannelController.cpp)
target_include_directories(tv_test PRIVATE include)
target_link_libraries(tv_test
  GTest::gtest_main GMock::gmock_main)
gtest_discover_tests(tv_test)
```

❏ settings.json & 필수 확장

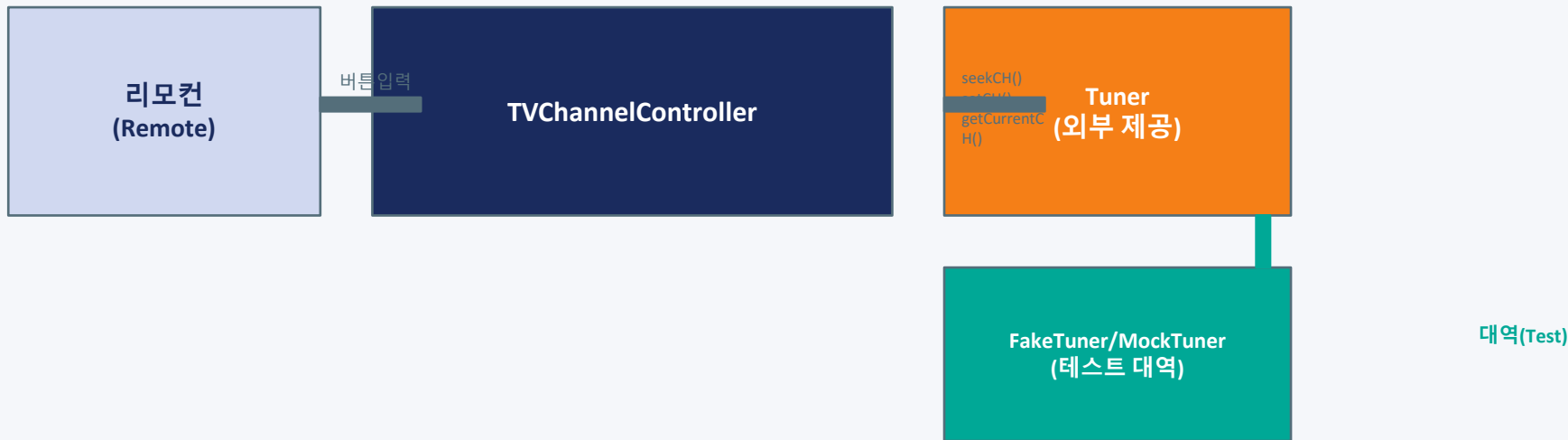
```
// .vscode/settings.json (공통 설정)
{
  "editor.formatOnSave": true,
  "editor.tabSize": 4,

  // C++ 설정
  "C_Cpp.default.cppStandard": "c++17",
  "cmake.buildDirectory":
    "${workspaceFolder}/build",
  "clangd.arguments": [
    "--header-insertion=never"
  ],

  // Java 설정 (병행 시)
  "java.compile.nullAnalysis.mode": "auto",
  "java.test.defaultConfig": "junit5"
}

// — VSCode 필수 확장 —————
// C++:
// • C/C++ Extension Pack (Microsoft)
// • CMake Tools
// • C++ TestMate
// • Coverage Gutters
// • clangd

// Java:
// • Extension Pack for Java
// • Test Runner for Java
// • Coverage Gutters
```



Tuner 인터페이스 (외부 제공 — 수정 불가)

메서드	반환값	설명	Java 시그니처	C++ 시그니처
<code>seekCH()</code>	string	현재 채널에서 증가 방향으로 시청 가능 채널 검색 후 반환	<code>String seekCH()</code>	<code>std::string seekCH()</code>
<code>setCH(ch)</code>	void	지정 채널값으로 현재 채널 변경 (범위: 0~99, 벗어난다면 예외)	<code>void setCH(String ch)</code>	<code>void setCH(std::string ch)</code>
<code>getCurrentCH()</code>	string	현재 설정된 채널값 반환	<code>String getCurrentCH()</code>	<code>std::string getCurrentCH()</code>

ITuner 인터페이스는 외부 제공 — 절대 수정 불가! FakeTuner(상태 기반) / MockTuner(GMock 행위 검증) 를 직접 구현

기능 명세 1 — 숫자 버튼 채널 변경

요구사항

S1-1 **Given** 리모컨 '1' 입력 후 '확인' **When** 확인 버튼 **Then** 1번 채널로 변경

S1-2 **Given** 리모컨 '1' → '2' 연속 입력 **When** 자동 변경 **Then** 12번 채널로 변경

S1-3 **Given** '1','2','3','4' 연속 입력 **When** 자동 변경 **Then** 12번 → 34번 순서로 변경

S1-4 **Given** '4','5','6' 연속 입력 후 다른 버튼 **When** 다른 버튼 **Then** 6 무효화 (영향 없음)

S1-5 **Given** '0','7' 연속 입력 **When** 자동 변경 **Then** 7번 채널로 변경

💡 경계값 주목

- 두 자리 입력 (12, 34): 반드시 별도 테스트 케이스 작성 — 자동 변경 타이밍 검증
- 무효화 (6): 세 자리 입력 후 다른 버튼 → `inputBuffer` 초기화 검증
- '0'으로 시작: '0'→'7' 입력 시 07 아닌 7번 채널 (두 번째 자리만 유효)
- 세 자리 이상 입력: 마지막 두 자리로 처리 ('1','2','3','4' → 12, then 34)

★ 선호 채널 (Favorite Channel)

S2-1 시청 중 '선호 채널' 버튼 → 현재 채널이 목록에 없으면 추가

S2-2 시청 중 '선호 채널' 버튼 → 이미 목록에 있으면 삭제 (토글)

S2-3 12→★ 08→★ 37→★ 08→★ 06→★ 순 입력 결과
목록: {6, 12, 37} (08은 추가 후 삭제됨)

12→★

✓12

08→★

✓08

37→★

✓37

08→★

✗08

06→★

✓06

▶▶ 다음 선호 채널 (Next Favorite)

S3-1 '다음선호채널' 버튼 → 목록 내 현재 채널보다 큰 값 중 가장 작은 채널로 이동

S3-2 목록 {1,4,12,56}, 현재 6번 시청 → '다음선호채널' 누르면 12번으로 변경

S3-3 목록 {1,4,12,56}, 현재 56번 시청 → '다음선호채널' 누르면 1번으로 변경 (wrap-around)

S3-4 선호 채널 목록이 비어있으면 채널 변경 없음

1

4

12

56

→ 1 (wrap)

Fake Object

실제 동작하는 단순 구현체.
상태(현재 채널)를 직접 관리.

장점

- ✓ 외부 라이브러리 불필요
- ✓ 동작 이해가 직관적
- ✓ C++ 환경에서 특히 유용

단점

- X 유지보수 필요
- X 복잡한 시나리오 구현 어려움

Mock (GMock / Mockito)

호출 검증 중심의 대역.
기대 행동을 선언적으로 정의.

장점

- ✓ 호출 횟수·순서 검증 가능
- ✓ 복잡한 시나리오에 강력
- ✓ C++: GMock, Java: Mockito

단점

- X 학습 곡선 존재
- X 과도한 검증은 테스트를 깨지기 쉽게 함



언제 어떤 것을 선택할까?

Fake 선택

상태 기반 로직 검증 (채널 값이 제대로 바뀌었는가?)
C++ 환경 우선 | 외부 라이브러리 없이 단순 구현

Mock 선택

행위 기반 검증 (setCH가 올바른 값으로 몇 번 호출됐는가?)
호출 순서/횟수 검증이 필요한 경우

이 과제에서

숫자버튼 → setCH 호출 검증: Mock 적합
선호채널 목록 상태 검증: Fake 적합

🔗 C++17 (Google Test / GMock)

```
// — ITuner.h — 순수 가상 인터페이스 (외부 제공, 수정 불가) —  
#pragma once  
#include <string>  
  
class ITuner {  
public:  
    virtual ~ITuner() = default;  
    virtual std::string seekCH() = 0;  
    virtual void setCH(std::string ch) = 0;  
    virtual std::string getCurrentCH() = 0;  
};
```

🔗 Java 17 (JUnit 5 / Mockito)

```
// ITuner.java (외부 제공, 수정 불가)  
public interface ITuner {  
    String seekCH();  
    void setCH(String ch);  
    String getCurrentCH();  
}
```


🔧 C++17 (Google Test / GMock)

```
// — FakeTuner.h — 테스트용 Fake 구현 (직접 작성) —————
#pragma once
#include "ITuner.h"
#include <vector>
#include <algorithm>
#include <stdexcept>

class FakeTuner : public ITuner {
    int current_ = 0;
    std::vector<int> available_;
public:
    explicit FakeTuner(std::vector<int> avail)
        : available_(std::move(avail)) {}

    // 현재보다 큰 채널 중 가장 작은 것 (없으면 wrap)
    std::string seekCH() override {
        auto it = std::find_if(
            available_.begin(), available_.end(),
            [&](int ch){ return ch > current_; });
        current_ = (it != available_.end())
            ? *it : available_.front();
        return std::to_string(current_);
    }

    void setCH(std::string ch) override {
        int v = std::stoi(ch);
        if (v < 0 || v > 99)
            throw std::invalid_argument(
                "채널 범위 초과: " + ch);
        current_ = v;
    }

    std::string getCurrentCH() override {
        return std::to_string(current_);
    }
};
```

🔧 Java 17 (JUnit 5 / Mockito)

```
// FakeTuner.java (직접 작성)
public class FakeTuner implements ITuner {
    private int currentChannel = 0;
    private List<Integer> availableChannels;

    public FakeTuner(List<Integer> available) {
        this.availableChannels = available;
    }

    @Override
    public String seekCH() {
        int next = availableChannels.stream()
            .filter(ch -> ch > currentChannel)
            .findFirst()
            .orElseGet(() ->
                availableChannels.get(0));
        currentChannel = next;
        return String.valueOf(next);
    }

    @Override
    public void setCH(String ch) {
        int v = Integer.parseInt(ch);
        if (v < 0 || v > 99)
            throw new IllegalArgumentException(
                "채널 범위 초과: " + ch);
        currentChannel = v;
    }

    @Override
    public String getCurrentCH() {
        return String.valueOf(currentChannel);
    }
}
```

C++17 (Google Test / GMock)

```
// TVChannelControllerTest.cpp
#include <gtest/gtest.h>
#include "TVChannelController.h"
#include "FakeTuner.h"

// — 공통 픽스처 (SetUp/TearDown 자동 호출) —————
class ControllerTest : public ::testing::Test {
protected:
    std::unique_ptr<FakeTuner> tuner;
    std::unique_ptr<TVChannelController> ctrl;
    void SetUp() override {
        tuner = std::make_unique<FakeTuner>(
            std::vector<int>{1, 4, 12, 56});
        ctrl = std::make_unique<
            TVChannelController>(*tuner);
    }
};

// — 기능 1: 숫자 버튼 채널 변경 —————
// S1-1: 한 자리 입력 + 확인
TEST_F(ControllerTest, PressNumber1ThenConfirm) {
    ctrl->pressNumber(1);    // Given
    ctrl->pressConfirm();    // When
    EXPECT_EQ("1", tuner->getCurrentCH()); // Then
}

// S1-2: 두 자리 자동 변경
TEST_F(ControllerTest, Press1Then2_AutoChange) {
    ctrl->pressNumber(1);
    ctrl->pressNumber(2);    // 두 자리 완성 → 자동
    EXPECT_EQ("12", tuner->getCurrentCH());
}
```

C++17 (Google Test / GMock)

```
// S1-3: 연속 두 자리 × 2회
TEST_F(ControllerTest, Press1234_TwoStageChange) {
    ctrl->pressNumber(1); ctrl->pressNumber(2); // → 12
    ctrl->pressNumber(3); ctrl->pressNumber(4); // → 34
    EXPECT_EQ("34", tuner->getCurrentCH());
}

// S1-4: 버퍼 무효화
TEST_F(ControllerTest, OtherButtonCancelsBuffer) {
    ctrl->pressNumber(4);
    ctrl->pressNumber(5);
    ctrl->pressNumber(6); // 3자리 → 무효화
    ctrl->pressOther();
    // 6은 무효화 → 채널 변화 없음
    EXPECT_EQ("0", tuner->getCurrentCH());
}

// S1-5: '0', '7' → 7번
TEST_F(ControllerTest, Zero7_SingleDigit7) {
    ctrl->pressNumber(0);
    ctrl->pressNumber(7);
    EXPECT_EQ("7", tuner->getCurrentCH());
}
```

⚙ C++17 (Google Test / GMock)

```
// — 기능 2: 선호 채널 토글 —————
TEST_F(ControllerTest, FavoriteAdd_NewChannel) {
    tuner->setCH("12");
    ctrl->pressFavorite();
    const auto& favs = ctrl->getFavoriteChannels();
    EXPECT_NE(favs.end(),
              std::find(favs.begin(), favs.end(), 12));
}

TEST_F(ControllerTest, FavoriteToggle_Remove) {
    tuner->setCH("12");
    ctrl->pressFavorite(); // 추가
    ctrl->pressFavorite(); // 삭제 (토글)
    const auto& favs = ctrl->getFavoriteChannels();
    EXPECT_EQ(favs.end(),
              std::find(favs.begin(), favs.end(), 12));
}

// S2-3: 토글 시나리오 전체
TEST_F(ControllerTest, FavoriteToggleScenario) {
    for (int ch : {12, 8, 37, 8, 6}) {
        tuner->setCH(std::to_string(ch));
        ctrl->pressFavorite();
    }
    const auto& favs = ctrl->getFavoriteChannels();
    // {6, 12, 37} 만 남아야 함
    EXPECT_EQ(3u, favs.size());
    EXPECT_NE(favs.end(),
              std::find(favs.begin(), favs.end(), 6));
    EXPECT_NE(favs.end(),
              std::find(favs.begin(), favs.end(), 12));
    EXPECT_NE(favs.end(),
              std::find(favs.begin(), favs.end(), 37));
}
```

⚙ C++17 (Google Test / GMock)

```
// — 기능 3: 다음 선호 채널 —————
TEST_F(ControllerTest, NextFavorite_Normal) {
    for (int ch:{1,4,12,56}) ctrl->addFavorite(ch);
    tuner->setCH("6");
    ctrl->pressNextFavorite();
    EXPECT_EQ("12", tuner->getCurrentCH());
}

TEST_F(ControllerTest, NextFavorite_WrapAround) {
    ctrl->addFavorite(1); ctrl->addFavorite(56);
    tuner->setCH("56");
    ctrl->pressNextFavorite();
    EXPECT_EQ("1", tuner->getCurrentCH());
}

TEST_F(ControllerTest, NextFavorite_EmptyList) {
    tuner->setCH("6");
    ctrl->pressNextFavorite();
    EXPECT_EQ("6", tuner->getCurrentCH()); // 변화 없음
}
```

C++17 (Google Test / GMock)

```
// MockTuner.h - GMock MOCK_METHOD 자동 생성
#pragma once
#include <gmock/gmock.h>
#include "ITuner.h"

class MockTuner : public ITuner {
public:
    MOCK_METHOD(std::string, seekCH, (), (override));
    MOCK_METHOD(void, setCH, (std::string ch), (override));
    MOCK_METHOD(std::string, getCurrentCH, (), (override));
};

// TVChannelControllerMockTest.cpp
#include <gtest/gtest.h>
#include <gmock/gmock.h>
#include "TVChannelController.h"
#include "MockTuner.h"
using ::testing::Return;
using ::testing::_;

class ControllerMockTest : public ::testing::Test {
protected:
    MockTuner mockTuner;
    std::unique_ptr<TVChannelController> ctrl;
    void SetUp() override {
        ctrl = std::make_unique<
            TVChannelController>(mockTuner);
    }
};
```

Java 17 (JUnit 5 / Mockito)

```
// Java - Mockito Mock 테스트
@ExtendWith(MockitoExtension.class)
class TVChannelControllerTest {

    @Mock ITuner tuner;
    TVChannelController controller;

    @BeforeEach
    void setUp() {
        controller =
            new TVChannelController(tuner);
    }

    @Test
    @DisplayName("숫자버튼 '1' 확인→setCH(1)호출")
    void pressNumber1ThenConfirm() {
        controller.pressNumber(1);
        controller.pressConfirm();
        verify(tuner).setCH("1");
    }

    @Test
    @DisplayName("'1', '2' 연속→setCH(12)호출")
    void press1Then2() {
        controller.pressNumber(1);
        controller.pressNumber(2);
        verify(tuner).setCH("12");
    }
}
```

C++17 (Google Test / GMock)

```
// 숫자버튼 '1' + 확인 → setCH("1") 정확히 1회 호출
TEST_F(ControllerMockTest, PressNumber1Confirm) {
    EXPECT_CALL(mockTuner, setCH("1")).Times(1);
    ctrl->pressNumber(1);
    ctrl->pressConfirm();
}

// '1','2' 연속 → setCH("12") 1회 호출
TEST_F(ControllerMockTest, Press1Then2_SetCH12) {
    EXPECT_CALL(mockTuner, setCH("12")).Times(1);
    ctrl->pressNumber(1);
    ctrl->pressNumber(2);
}

// pressFavorite → getCurrentCH() 호출 검증
TEST_F(ControllerMockTest, PressFavorite_GetsCurrentCH) {
    EXPECT_CALL(mockTuner, getCurrentCH())
        .WillOnce(Return("12"));
    ctrl->pressFavorite();
}

// pressNextFavorite: 선호목록 있을 때 setCH 호출
TEST_F(ControllerMockTest, NextFav_CallsSetCH) {
    ctrl->addFavorite(12); ctrl->addFavorite(56);
    EXPECT_CALL(mockTuner, getCurrentCH())
        .WillOnce(Return("6"));
    EXPECT_CALL(mockTuner, setCH("12")).Times(1);
    ctrl->pressNextFavorite();
}
```

Java 17 (JUnit 5 / Mockito)

```
@Test
@DisplayName("선호채널추가:미등록채널")
void addFavoriteChannel() {
    when(tuner.getCurrentCH())
        .thenReturn("12");
    controller.pressFavorite();
    assertTrue(controller
        .getFavoriteChannels()
        .contains(12));
}

@Test
@DisplayName("다음선호채널:wrap-around")
void nextFavoriteWrapAround() {
    controller.addFavorite(1);
    controller.addFavorite(56);
    when(tuner.getCurrentCH())
        .thenReturn("56");
    controller.pressNextFavorite();
    verify(tuner).setCH("1");
}
}
```

TVChannelController 설계 가이드 & 구현 힌트

구현 힌트

C++17 (Google Test / GMock)

```
// TVChannelController.h
#pragma once
#include "ITuner.h"
#include <vector>
#include <algorithm>
#include <string>

class TVChannelController {
    ITuner& tuner_;           // 참조로 주입 (인터페이스)
    int inputBuffer_ = -1;    // -1: 버퍼 비어있음
    std::vector<int> favorites_; // 정렬 유지

public:
    explicit TVChannelController(ITuner& t)
        : tuner_(t) {}

    void pressNumber(int digit);    // 숫자 버튼
    void pressConfirm();           // 확인 버튼
    void pressFavorite();          // 선호채널 토글
    void pressNextFavorite();      // 다음 선호채널
    void pressOther();             // 버퍼 무효화

    const std::vector<int>&
    getFavoriteChannels() const { return favorites_; }

    // 테스트 헬퍼
    void addFavorite(int ch) {
        if (!isFavorite(ch)) {
            favorites_.push_back(ch);
            std::sort(favorites_.begin(),
                      favorites_.end());
        }
    }
}
```

Java 17 (JUnit 5 / Mockito)

```
// TVChannelController.java
public class TVChannelController {
    private final ITuner tuner;
    private int inputBuffer = -1;
    private List<Integer> favorites
        = new ArrayList<>();

    public TVChannelController(ITuner tuner){
        this.tuner = tuner;
    }

    public void pressNumber(int digit) {
        if (inputBuffer == -1) {
            inputBuffer = digit;
        } else {
            int ch = inputBuffer * 10 + digit;
            inputBuffer = -1;
            applyChannel(ch);
        }
    }

    public void pressConfirm() {
        if (inputBuffer != -1) {
            applyChannel(inputBuffer);
            inputBuffer = -1;
        }
    }

    public void pressOther(){
        inputBuffer = -1;
    }
}
```

TVChannelController 설계 가이드 & 구현 힌트

구현 힌트

🔗 C++17 (Google Test / GMock)

```
private:
    bool isValidChannel(int ch) const {
        return ch >= 0 && ch <= 99;
    }
    bool isFavorite(int ch) const {
        return std::find(favorites_.begin(),
                        favorites_.end(), ch) != favorites_.end();
    }
    void applyChannel(int ch);
};

// TVChannelController.cpp
void TVChannelController::pressNumber(int digit) {
    if (inputBuffer_ == -1) {
        inputBuffer_ = digit;    // 첫 자리 저장
    } else {
        int ch = inputBuffer_ * 10 + digit;
        inputBuffer_ = -1;
        applyChannel(ch);        // 두 자리 완성 → 자동 변경
    }
}

void TVChannelController::pressConfirm() {
    if (inputBuffer_ != -1) {
        int ch = inputBuffer_;
        inputBuffer_ = -1;
        applyChannel(ch);
    }
}

void TVChannelController::pressOther() {
    inputBuffer_ = -1;           // 버퍼 무효화
}
```

🔗 Java 17 (JUnit 5 / Mockito)

```
public void pressFavorite() {
    int ch = Integer.parseInt(
        tuner.getCurrentCH());
    if (favorites.contains(ch))
        favorites.remove((Integer)ch);
    else {
        favorites.add(ch);
        Collections.sort(favorites);
    }
}

public void pressNextFavorite() {
    if (favorites.isEmpty()) return;
    int cur = Integer.parseInt(
        tuner.getCurrentCH());
    int next = favorites.stream()
        .filter(ch -> ch > cur)
        .findFirst()
        .orElse(favorites.get(0));
    applyChannel(next);
}

private void applyChannel(int ch){
    if (ch<0||ch>99) throw new
        IllegalArgumentException();
    tuner.setCH(String.valueOf(ch));
}

public List<Integer> getFavoriteChannels(){
    return Collections
        .unmodifiableList(favorites);
}
}
```

🔗 C++17 (Google Test / GMock)

```
void TVChannelController::pressFavorite() {
    int ch = std::stoi(tuner_.getCurrentCH());
    if (isFavorite(ch)) {
        favorites_.erase(std::remove(
            favorites_.begin(), favorites_.end(), ch),
            favorites_.end());
    } else {
        favorites_.push_back(ch);
        std::sort(favorites_.begin(), favorites_.end());
    }
}

void TVChannelController::pressNextFavorite() {
    if (favorites_.empty()) return;
    int cur = std::stoi(tuner_.getCurrentCH());
    auto it = std::upper_bound(
        favorites_.begin(), favorites_.end(), cur);
    int next = (it != favorites_.end())
        ? *it : favorites_.front();
    applyChannel(next);
}

void TVChannelController::applyChannel(int ch) {
    if (!isValidChannel(ch))
        throw std::invalid_argument("Invalid ch");
    tuner_.setCH(std::to_string(ch));
}
```




Red

실패하는 테스트 작성



Green

최소 코드로 테스트 통과



Refactor

중복 제거 & 가독성 향상



커밋 메시지 가이드 (TDD 과정이 드러나게)

Red

`git commit -m "test: 숫자버튼_1_확인_채널변경_실패"`

Green

`git commit -m "feat: 숫자버튼_1_확인_채널변경_구현"`

Red

`git commit -m "test: 두자리숫자_12_채널변경_실패"`

Green

`git commit -m "feat: 두자리숫자_입력버퍼_구현"`

Red

`git commit -m "test: 선호채널_추가_삭제_토글_실패"`

Green

`git commit -m "feat: 선호채널_토글_구현"`

Refactor

`git commit -m "refactor: 채널범위_검증_헬퍼_메서드_추출"`

Red

`git commit -m "test: 다음선호채널_wraparound_실패"`

Green

`git commit -m "feat: 다음선호채널_wraparound_구현"`

Refactor

`git commit -m "refactor: favorites_정렬_보장_로직_정리"`

커밋 빈도: 각 Red/Green/Refactor 단계마다 push → TDD 과정이 커밋 히스토리에 명확히 드러나야 합니다! (평가 핵심 ★★★)

숫자 버튼 채널 변경

- ☐ 한 자리 숫자 입력 + 확인 → 채널 변경
- ☐ 두 자리 숫자 입력 → 자동 변경
- ☐ 세 자리 이상 → 마지막 두 자리로 처리
- ☐ '0'으로 시작 → 두 번째 자리만 유효
- ☐ 기타 버튼 입력 → 버퍼 무효화
- ☐ 유효하지 않은 채널 → 예외 처리

선택 채널

- ☐ 미등록 채널 → 목록 추가
- ☐ 이미 등록된 채널 → 목록 삭제
- ☐ 여러 채널 순서대로 추가/삭제
- ☐ 목록 정렬 순서 검증
- ☐ 최대 저장 개수 경계 (있는 경우)
- ☐ 목록 조회 (불변 반환 확인)

다음 선택 채널

- ☐ 목록 내 현재보다 큰 채널로 이동
- ☐ 가장 큰 채널 → 가장 작은 채널 (wrap)
- ☐ 현재 채널이 목록 외 값일 때
- ☐ 목록이 비어있을 때 → 채널 변경 없음

Tuner Test Double

- ☐ 초기 채널값 0~99 범위 검증
- ☐ setCH() 유효 값 → 정상 동작
- ☐ setCH() 유효하지 않은 값 → 예외
- ☐ seekCH() 10회 연속 → 유효 채널 반환
- ☐ seekCH() wrap-around 동작 검증
- ☐ ... (추가 테스트 케이스)

⚙ C++17 - CMake + lcov

```
# — C++ - CMake + Google Test —————
# 빌드 & 테스트 전체 실행
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j4
cd build && ctest --output-on-failure

# 특정 테스트만 실행 (필터)
ctest -R "PressNumber" --output-on-failure
./tv_test --gtest_filter="ControllerTest.Press*"

# — 커버리지 측정 (gcov / lcov) —————
cmake -B build -DCMAKE_BUILD_TYPE=Debug -DCMAKE_CXX_FLAGS="--coverage" -
DCMAKE_EXE_LINKER_FLAGS="--coverage"
cmake --build build && cd build && ctest
lcov --capture --directory . --output-file coverage.info
genhtml coverage.info --output-directory cov/
# 브라우저에서 확인: open cov/index.html

# — VSCode CMake Tools 활용 —————
# ① CMake Tools 확장 설치 후
# 하단 상태바 [Build Kit] 선택
# ② Ctrl+Shift+P → "CMake: Build"
# ③ Ctrl+Shift+P → "CMake: Run Tests"
# ④ C++ TestMate 확장 → Testing 탭에서
# 개별 테스트 ▶ 클릭 실행
# ⑤ Coverage Gutters 확장 →
# gcov 결과로 라인별 커버리지 색상 표시
```

C++ — TestMate & Coverage Gutters

- ① CMake Tools: 빌드·테스트 자동화 (상태바 클릭)
- ② C++ TestMate: Testing 탭 연동, 단건 실행
- ③ Coverage Gutters: gcov 결과 라인별 색상 표시

⚙ Java 17 - Gradle / Maven + JaCoCo

```
# Java - Gradle + JUnit 5
# 터미널 (VSCode 내장 Ctrl+backtick)

# Gradle 테스트 실행
./gradlew test

# 커버리지 리포트 생성 (JaCoCo)
./gradlew test jacocoTestReport

# 리포트 확인
# build/reports/jacoco/test/html/index.html

# 특정 테스트만 실행
./gradlew test --tests "/*.숫자버튼*"

# Maven 사용 시
mvn test
mvn jacoco:report

# VSCode Testing 탭 활용
# ① Ctrl+Shift+P → "Test: Run All Tests"
# ② 사이드바 (Testing) → 테스트 트리
# ③ 개별 테스트 옆 ▶ 버튼으로 단건 실행
# ④ Coverage Gutters 확장으로
# 라인별 커버리지 색상 표시
```

Java — Test Runner & Coverage Gutters

- ① Test Runner for Java: Testing 탭 연동
- ② 개별 테스트 옆 ▶ 버튼으로 단건 실행
- ③ Coverage Gutters: JaCoCo 결과 라인별 표시



제출 체크리스트

☐ Repository: TDD_TV_XX (개인번호 포함)

☐ description 에 Author 성명 기재

☐ main → dev → refactoring 브랜치 구조 유지

☐ 기존 branch 삭제 금지, merge 금지

☐ TDD 사이클 커밋 히스토리 확인 가능

☐ FakeTuner 또는 MockTuner 구현 포함

☐ 테스트 커버리지 80% 이상 달성

☐ 모든 테스트 Green 상태로 최종 push



평가 항목

TDD 커밋 히스토리

★★★★

Red→Green→Refactor 사이클이 커밋에 드러나는가

테스트 커버리지

★★★★

경계값·예외 케이스 포함 80% 이상

Test Double 활용

★★★☆☆

Fake/Mock 으로 Tuner 의존성 격리

테스트 가독성

★★★☆☆

Given-When-Then, 이름 명확성

코드 품질

★★★☆☆

단일 책임, 중복 제거, 클린코드

Write the test first.

- ✓ Tuner 는 믿어라 — 하지만 테스트에선 Fake/Mock 으로 대체하라
- ✓ 커밋 하나하나가 TDD 증거 — Red→Green→Refactor 를 남겨라
- ✓ 경계값을 빠짐없이 — 0, 99, 빈 목록, wrap-around
- ✓ Java 와 C++ 은 도구가 다를 뿐, 설계 원칙은 같다
- ✓ C++: GoogleTest/GMock | Java: JUnit5/Mockito — 모두 VSCode 에서 완벽 동작

GOOD LUCK

